

From Pipeline to Agent: Voyage Cancellations Agent

The Next Challenge

In Session 1 you built a Travel Policy Assistant that can answer questions about Voyage's refund and cancellation rules. The support team is pleased — but they have a new request.

They don't just want the assistant to *answer* questions. They want it to *act*: look up a booking, check whether it is eligible for a refund, and process the cancellation if it is. The current pipeline cannot do any of this — it can only retrieve text and generate a response.

Our Task

A RAG pipeline is designed to answer questions, but an agent is designed to take actions — and when a system needs to make decisions, call APIs, and complete workflows, we need an agent rather than a static retrieval pipeline. Our task today is to convert our RAG pipeline into an agentic flow.

Smolagents vs LangGraph: Prototype and System Design

There are multiple ways to build agents in Python, and the choice often depends on the level of structure and control you need.

Smolagents is designed to make it very easy to give a model access to tools. With minimal setup, you can prototype task-oriented agents quickly. It works well when you want to experiment with tool use, explore automation ideas, or build lightweight workflows without managing the underlying execution loop yourself.

LangGraph, by contrast, makes the reasoning loop explicit. It allows you to define how tools are orchestrated, how memory is handled, and how execution progresses from step to step. This makes it better suited for more structured systems, especially when you want visibility into intermediate reasoning or need tighter control over behavior.

In practice, smolagents is often a strong choice for rapid prototyping, while LangGraph provides more flexibility for building structured, multi-step workflows. As systems grow in complexity — especially when retrieval, memory, and multi-tool coordination are involved — having explicit control over orchestration becomes increasingly important.

Rebuild the Policy Retriever

We rebuild the vector store from Session 1 so this notebook runs standalone. The agent will call this via a `lookup_policy` tool.

Build Your First Tool

Tools are standard Python functions decorated with `@tool`. The **docstring** is what the model reads to decide when to call the tool — a clear, specific docstring leads to better decisions than a vague one.

We start with a single tool that wraps the Session 1 retriever.

Tool 1: The Policy Lookup This tool gives the Agent access to our RAG retriever. We write a broad docstring so the Agent knows it can use this for any policy question.

Wire the Agent

We now combine the language model and our tools into a single reasoning loop. The agent follows the ReAct pattern (Reason + Act): it thinks about what to do, selects a tool if needed, observes the result, and continues reasoning until it can produce a final answer. Unlike a traditional function call, this loop allows the model to dynamically decide the next step based on new information it receives from tools.

Importantly, the agent is not simply calling a tool once. It can call multiple tools in sequence, using the output of one step to inform the next. This ability to reason over intermediate results is what makes agents powerful for workflow automation.

Test the Policy Agent

With `verbose=True` you will see every Thought, Action, and Observation printed as the agent runs. Watch the reasoning chain — this is the ReAct loop in action.

Expanding the Toolkit

Policy lookup alone is not enough. To process a cancellation the agent also needs to:

1. **Look up the booking** to find out the ticket type.
2. **Cancel the ticket** once the policy confirms it is eligible.

We add two more tools. Notice how the docstrings guide the agent's decision on *when* to use each one.

Wire the Full Agent

We now rebuild the agent with the new workflow.

Persistent Memory

Up to this point, each request has been treated as an independent interaction. The agent has no awareness of previous messages. In real applications, however, users ask follow-up questions that rely on earlier context. To support this, we attach a memory layer to the agent.

In LangGraph, memory is organized by `thread_id`. Each thread represents a separate conversation. Messages within the same thread share context. Messages in different threads remain completely isolated.

##Fixing the lookup Policy tool